VIP – AI Hub for Config-Driven Agent Lifecycle Management, Orchestration, and Observability

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Ibtihel Mnaja

Encadrant Entreprise/ Business site Supervisor
• Nom & Prénom /Name & Surname : Ahmed Rebai **Cachet & Signature / Stamp & Signature**

Encadrant Académique/Academic Supervisor
• Nom & Prénom /Name & Surname : Jihen Jebri **Signature / Signature**

Ce formulaire doit être rempli, signé et **scanné**/This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover page.

ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn
Siége Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889
Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454

esprit
Se former autrement

2025 - 2026
GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : Data Science

VIP — Agentic AI Hub for Config-Driven Agent Lifecycle Management and Orchestration

By: *Ibtihel Mnaja*

Academic supervisor: *Jihen Jebri*

Corporate Internship Supervisor: *Ahmed Rebai*

VALUE

Abstract

Agentic Artificial Intelligence refers to AI systems capable of performing tasks through autonomous or semi-autonomous agents that can reason, use tools, interact with external systems, and collaborate within structured workflows. In the financial services industry, the adoption of such systems requires not only advanced AI capabilities, but also strong guarantees in terms of scalability, control, auditability, and operational reliability.

This internship report presents the design and implementation of an **AI Agentic Hub Platform** aimed at building, deploying, and operating intelligent applications at scale. The proposed platform is based on a modular multi-agent architecture in which agents are generated from declarative configurations, deployed as independent containerized services, and coordinated through an orchestration layer. This approach makes it possible to create specialized agents without modifying their source code, by relying on configurable prompts, tools, model parameters, and input/output schemas.

The work focuses on several key aspects of enterprise AI deployment, including agent generation, workflow orchestration, schema-based communication, prompt and artifact versioning using MLflow, Docker-based containerization, and observability. The objective is to provide a controlled and extensible blueprint for developing financial AI applications while reducing fragmentation, improving governance, and ensuring traceability across the full lifecycle of intelligent systems.

Keywords: Agentic AI, Multi-Agent Systems, AI Orchestration, Docker, MLflow, Financial Services, Schema Validation, Observability.

Summary

This internship project focuses on the design and implementation of **VIP – Value Intelligent Platform**, an **Enterprise-Grade Agentic AI Hub** aimed at building, deploying, and operating intelligent applications at scale.

- **Dynamic Agent Generation:** Developed a configurable Agent Factory capable of creating specialized AI agents from declarative configurations, without modifying the source code.
- **Modular and Containerized Architecture:** Designed agents as independent FastAPI services deployed in isolated Docker containers, ensuring scalability, separation of concerns, and easier maintenance.
- **Centralized Governance and Versioning:** Integrated MLflow to manage prompts, schemas, and configuration artifacts, enabling traceability, version control, and a "Zero Code Change" approach.
- **Intelligent Orchestration:** Implemented an orchestration layer supported by an adapter to coordinate multi-agent workflows and manage schema transformations between agents.

Conclusion: The platform provides a scalable and reusable blueprint for enterprise AI applications, allowing organizations to transform complex business processes into modular, auditable, and intelligent agent-based workflows.

Acknowledgments

I would first like to express my sincere gratitude to the entire team at **Value Digital Services** for welcoming me into such a professional, collaborative, and innovative environment. The experience gained throughout this internship has been both enriching and inspiring, and I truly appreciated the support and positive atmosphere within the company.

I am deeply grateful to my professional supervisor, Mr. **Ahmed REBAI**, for his guidance, technical expertise, and continuous support throughout this project. His vision, advice, and constructive feedback played a major role in the successful realization of the *VIP – Value Intelligent Platform*.

I would also like to sincerely thank my academic supervisor, Ms. **Jihen JERBI**, for her valuable supervision, encouragement, and academic guidance. Her support helped ensure the quality and rigor of this work throughout the internship period.

On a personal note, I would like to express my deepest gratitude to my parents, **Hayet** and **Ali**, for their unconditional love, sacrifices, and constant encouragement throughout my studies. Their trust and support have always been my greatest source of motivation.

I would also like to warmly thank my sister **Dhouha**, as well as my brothers **Borhen** and **Louey**, for their continuous encouragement, patience, and support during both the challenging and rewarding moments of this journey.

Finally, I would like to thank everyone who contributed, directly or indirectly, to the success of this internship experience and to the completion of this project. This experience has been a major step in both my academic and professional journey, and I am truly grateful for the knowledge and experience I have gained.

Table of Contents

List of Figures

List of Tables

Glossary

ADLC:	Agent Development Lifecycle
AI:	Artificial Intelligence
API:	Application Programming Interface
IBM:	International Business Machines
LLM:	Large Language Model
QA:	Quality Assurance
REST:	Representational State Transfer
UI:	User Interface
UX:	User Experience
VIP:	Value Intelligent Platform

Introduction

The recent evolution of Artificial Intelligence has marked a major shift from traditional conversational models toward *agentic systems*. These systems are characterized by their ability to reason, use external tools, interact with data sources, and execute complex multi-step workflows with a certain level of autonomy. In this context, the subject of this internship focuses on the design and implementation of **VIP – Value Intelligent Platform**, an enterprise-grade Agentic AI Hub intended to build, deploy, and operate intelligent applications at scale across different business domains.

Despite the significant progress of Large Language Models, their integration into enterprise environments remains challenging. The main difficulty lies in transforming AI prototypes into reliable, reusable, and governable systems that can be deployed in real operational contexts. Traditional approaches often rely on hard-coded agents, making them difficult to adapt, maintain, or extend. The problem addressed in this project is therefore to design a modular platform capable of generating specialized agents from declarative configurations, while ensuring scalability, traceability, and operational control.

The adopted approach is based on a decoupled and containerized multi-agent architecture. Each agent is designed as an independent FastAPI service, deployed in a Docker container, and configured through prompts, tools, model parameters, and input/output schemas. MLflow is used to manage and version configuration artifacts such as prompts and schemas, while an orchestration layer coordinates the execution of agent workflows. A semantic adapter is also introduced to manage schema transformations between agents and ensure smooth communication across the platform.

This report is organized into five chapters detailing the progression of the work carried out during the internship. **Chapter 1** presents the general context of the internship and the host company. **Chapter 2** introduces the preliminary work completed during the onboarding phase. **Chapter 3** presents the overall architecture of the VIP platform. **Chapter 4** details the design and implementation of the agent generation mechanism and the core platform components. **Chapter 5** focuses on orchestration, observability, and evaluation aspects. Finally, a **conclusion** summarizes the main contributions of the project and proposes possible future improvements.

Chapter 1

Introduction and Project Context

1.1 Introduction

The rapid adoption of AI agents and generative language models, including large language models (LLMs) and small language models (SLMs), is creating new opportunities for enterprise automation while increasing the need for reliable orchestration, deployment, and observability. In this context, my internship at **Value Digital Services** focused on building a modular, config-driven platform for agent generation and management, alongside the evaluation of small language models through SLM-evals. This chapter introduces the host company, clarifies the project goals and problem context, and presents the methodology followed throughout the internship.

1.2 Host Company and Areas of Expertise

Value Digital Services is a Tunisian company founded in 2019 and specialized in digital transformation and AI consulting. Its activities combine software engineering, data engineering, artificial intelligence, and strategic consulting, which made it an appropriate environment for an internship focused on building an enterprise AI agent platform and evaluating small language models.

Figure 1: Logo of Value Digital Services

- **Digital Factory:** software engineers, DevOps engineers, QA engineers, business analysts, and UX/UI designers build and deliver modern digital products.
- **Data Factory:** data engineers and data analysts design data pipelines, analytics workflows, and data platforms that support decision-making.

- **AI Factory:** AI researchers and machine learning engineers design and deploy AI-driven solutions, including generative AI and intelligent automation systems.
- **Advisory:** consultants support clients in structuring digital transformation initiatives, aligning technical choices with business and governance needs.

These poles work together to deliver modern digital solutions, from consulting and design to data infrastructure, AI implementation, deployment, and operational support.

1.3 Project Overview

The project, entitled **VIP – Value Intelligent Platform**, aims to design and implement an enterprise-grade AI agent orchestration platform capable of building, deploying, and managing intelligent applications through a modular multi-agent architecture. Unlike traditional monolithic AI systems, the platform follows an *Agent-as-a-Service* approach, where specialized agents are dynamically generated from declarative configurations and deployed as independent containerized services.

VIP provides a unified environment for agent creation, workflow orchestration, observability, and lifecycle management. Users can configure agents by selecting models, prompts, schemas, providers, and tools without directly modifying the source code. The generated agents can then be integrated into multi-agent execution pipelines coordinated through an orchestration layer and monitored through built-in observability services.

This platform directly addresses the gaps identified in existing solutions by combining support for multiple LLM providers and open-source models with a modular architecture, declarative configuration management, and built-in observability. This design reduces dependency on a single proprietary provider, simplifies agent creation and deployment, and makes agent behavior easier to trace, evaluate, and improve over time.

Alongside VIP, I developed the **SLM-evals** side project, a reproducible evaluation pipeline for benchmarking LLM and SLM providers. It compares OpenAI, Groq, Phi-4, Llama 3.1, and other alternatives under controlled prompts and golden datasets, while tracking latency, token usage, and layered evaluation metrics. The results of this pipeline guide the choice of models for the *LLM-provider* layer and support VIP's objective of maintaining provider flexibility while selecting models that fit performance, cost, and reliability requirements.

1.4 Problem Statement

Despite the rapid evolution of AI agents and large language models, the deployment of agentic systems in enterprise environments remains complex. These challenges matter because enterprises need systems that can scale reliably, remain auditable, control operational costs, and comply with internal governance and security requirements. When agent platforms lack provider flexibility, observability, automation, or modular orchestration, they become difficult to deploy at scale and hard to maintain in production.

The main challenges addressed by this project can be summarized as follows:

- **High Operational Costs and Vendor Lock-in:** Existing AI agent solutions can be expensive to operate at scale, especially when they depend heavily on proprietary LLM APIs. This dependence increases costs, limits provider flexibility, and makes it harder to switch between OpenAI, Groq, Azure, open-source LLMs, or SLM alternatives.

- **Poor Observability:** Many current systems lack sufficient traceability, monitoring, and execution tracking. This limits debugging, performance evaluation, and the audit evidence required for compliance and governance.

- **Absence of Automated Deployment:** Creating and deploying new agents is often a manual and time-consuming process. Without automated deployment mechanisms, it becomes difficult to rapidly generate, configure, and launch agents in a consistent way.

- **Lack of Orchestration and Modularity:** Existing approaches often do not provide strong workflow orchestration capabilities. Agents are not always designed as reusable and modular components, which limits workflow composition and makes multi-agent collaboration harder to manage.

Based on these limitations, the central problem of this project can be formulated as follows:

> How can we design a modular, secure, observable, and cost-optimized AI agent orchestration platform that enables dynamic agent creation, automated deployment, and workflow composition while evaluating and integrating multiple LLM and SLM providers to reduce vendor lock-in, control costs, and maintain strong performance?

.

1.5 Objectives

Based on the problem statement, the objectives of the project are divided into functional and non-functional objectives.

Functional Objectives

- Enable dynamic agent creation from declarative YAML or JSON configuration files.

- Provide a visual workflow builder for composing and managing multi-agent execution pipelines.

- Offer a unified interface for selecting models, prompts, schemas, providers, and tools without modifying the source code.

- Support reusable multi-agent pipelines that can combine specialized agents into structured workflows.

- Integrate SLM-evals results into the provider selection process to compare LLM and SLM options before deployment.

Non-functional Objectives

- Reduce vendor lock-in and operational cost by evaluating multiple providers, including proprietary LLM APIs and local SLM alternatives.
- Optimize deployment costs through containerized, configurable, and reusable agent services.
- Ensure observability and traceability by monitoring execution traces, token usage, latency, and estimated cost.
- Preserve modularity and scalability by allowing agents, tools, providers, and workflows to evolve independently.
- Strengthen security and provider independence by limiting hard-coded provider dependencies and supporting controlled configuration management.

1.6 Technology Choices

The project relies on a modular technology stack selected to support agent generation, orchestration, deployment, observability, and provider flexibility.

1.6.1 Development Frameworks and Dev Environment

- **FastAPI:** lightweight Python framework used to build high-performance asynchronous APIs for agents and platform services.

Figure 2: Logo of FastAPI

- **Next.js:** React-based framework used to build the web frontend and provide a modular interface for agents, workflows, and configurations.

Figure 3: Logo of Next.js

- **LangChain:** library used to construct LLM-driven agents, connect tools, and manage prompt-based application logic.

Figure 4: Logo of LangChain

- **LangGraph:** library used to define graph-based orchestration logic for stateful multi-agent workflows.

Figure 5: Logo of LangGraph

- **Jupyter Notebook:** interactive environment used during prototyping, experimentation, SLM-evals, and prompt engineering.

Figure 6: Logo of Jupyter Notebook

1.6.2 LLM and SLM Providers

- **OpenAI:** commercial LLM provider used to access GPT-series models for high-quality generation and evaluation tasks.

Figure 7: Logo of OpenAI

- **Groq:** commercial inference provider used to access fast LLM serving, including Llama-based models.

Figure 8: Logo of Groq

- **Microsoft Phi-4:** small language model considered for cost-efficient and potentially self-hosted inference.

Figure 9: Logo of Microsoft Phi-4

- **Meta Llama :** open-weight LLM used as an alternative provider option for flexible deployment and benchmarking.

Figure 10: Logo of Meta Llama

- **SLM-evals:** used to benchmark providers and models in order to choose the best mix for the VIP provider layer.

1.6.3 Database, Storage, and Experiment Tracking

- **PostgreSQL:** relational database used to store configurations, metadata, workflows, and other structured platform data.

Figure 11: Logo of PostgreSQL

- **MLflow:** experiment tracking and artifact registry used to version models, prompts, schemas, datasets, and evaluation metrics.

Figure 12: Logo of MLflow

1.6.4 Deployment and Containerization

- **Docker:** container platform used to package individual agents, the builder, the orchestrator, and supporting services into reproducible images.

Figure 13: Logo of Docker

- **Docker Compose:** used during development and testing to run multi-service setups with consistent networking and configuration.

Figure 14: Logo of Docker Compose

1.6.5 Observability and Monitoring

- **Laminar:** used to capture execution traces, token usage, latency, and cost metrics across agents.

Figure 15: Logo of Laminar

1.7 Methodology

1.7.1 Methodological Approaches Considered

Several methodological approaches were considered in order to structure the development of the VIP platform. The project combines software engineering, data science, agent lifecycle management, orchestration, deployment, and observability. Therefore, the selected methodology had to support both iterative development and long-term architectural control. The approaches considered were Scrum, IBM Data Science Methodology, and IBM Agent Development Lifecycle (ADLC).

1.7.2 Scrum Agile Methodology

Scrum is an Agile framework based on iterative and incremental development. The official Scrum Guide describes Scrum as an iterative and incremental approach used to optimize predictability and control risk. It supports progressive delivery by organizing work into short cycles, reviewing progress regularly, and adapting the next development steps according to feedback.

Scrum was considered because the project required frequent adjustments, progressive development, and regular validation of features such as agent creation, workflow orchestration, prompt management, and observability dashboards.

However, Scrum alone was not sufficient for this internship because the project required more than sprint-based feature delivery. It also required a structured lifecycle for AI/data-related work, architectural traceability, governance, deployment control, and continuous monitoring of AI agents. For this reason, Scrum-inspired practices were retained for implementation organization, but Scrum was not selected as the main project methodology.

1.7.3 IBM Data Science Methodology – The Foundational Methodology

The IBM Data Science Methodology was considered as the foundational methodology because it provides the macro lifecycle of the project. The CognitiveClass data science methodology is structured around phases that move from business understanding and analytic approach to data requirements, collection, understanding, preparation, modeling, evaluation, deployment, and feedback. This structure makes it suitable for organizing an AI-oriented project from problem definition to operational feedback.

The methodology consists of 10 stages organized in a circular workflow:

- **Business Understanding:** defining the problem to be solved and establishing the main objectives.
- **Analytic Approach:** determining the appropriate analytical and technical approach to address the problem.
- **Data Requirements:** identifying the information and resources required to address the problem.
- **Data Collection:** gathering the required data and project inputs from the relevant sources.
- **Data Understanding:** exploring and examining the available information in order to understand quality, constraints, and relationships.
- **Data Preparation:** cleaning, transforming, and formatting the required data or resources for later use.
- **Modeling:** building predictive, descriptive, or intelligent models using appropriate algorithms and implementation choices.
- **Evaluation:** assessing the obtained results against the initial objectives and expected quality criteria.
- **Deployment:** implementing the solution in a production or operational environment.
- **Feedback:** monitoring performance, collecting feedback, and iterating on earlier stages when improvements are required.

Figure 16 presents the original IBM Data Science Methodology lifecycle, which was used as the main macro-framework for structuring the project from business understanding to deployment and feedback.

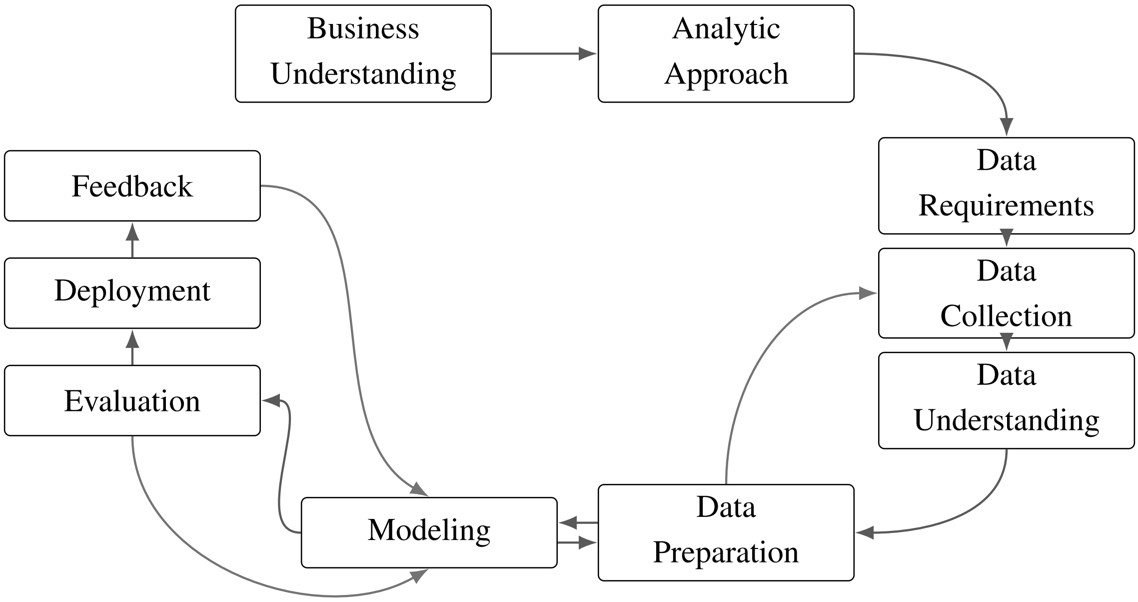

Figure 16: Original IBM Data Science Methodology lifecycle

This methodology is relevant for the VIP project because it supports a structured understanding of the business problem, the identification of requirements, the preparation of resources, the evaluation of outputs, and the deployment of the final solution. It also provides a feedback loop, which is important for improving agent behavior, model selection, and workflow reliability over time.

1.7.4 IBM Agent Development Lifecycle

While the IBM Data Science Methodology provides the overall project structure, it does not fully describe the specific lifecycle of AI agents. The VIP platform required a more agent-specific perspective because agents are generated from configuration, connected to tools, deployed as services, monitored, and improved after execution. For this reason, ADLC was used as a complementary framework.

IBM defines ADLC as a structured lifecycle for building, deploying, optimizing, and managing AI agents through distinct phases and iterative loops. In the context of VIP, this perspective is relevant because agents must remain reliable, observable, auditable, and aligned with business requirements after deployment.

In this report, the ADLC perspective is summarized through the following phases:

- **Plan:** alignment of stakeholders, use cases, objectives, success metrics, expected agent behavior, and operating procedures.
- **Code & Build:** implementation of agents through model selection, prompt design, tool integration, orchestration logic, version control, and controlled development environments.
- **Test & Release:** evaluation of agents against predefined criteria, including policy checks, security validation, and release readiness before cataloging or production use.

- **Deploy:** controlled rollout of validated agents into production environments with governance, versioning, rollback, and runtime security mechanisms.
- **Operate:** continuous observation and optimization of deployed agents using operational metrics such as accuracy, latency, cost, and user satisfaction.
- **Monitor:** ongoing auditing of agents and tools to support fairness, transparency, compliance, observability, and reproducibility.

Figure 17 illustrates the IBM Agent Development Lifecycle, which was used to guide the agent-specific phases of the project.

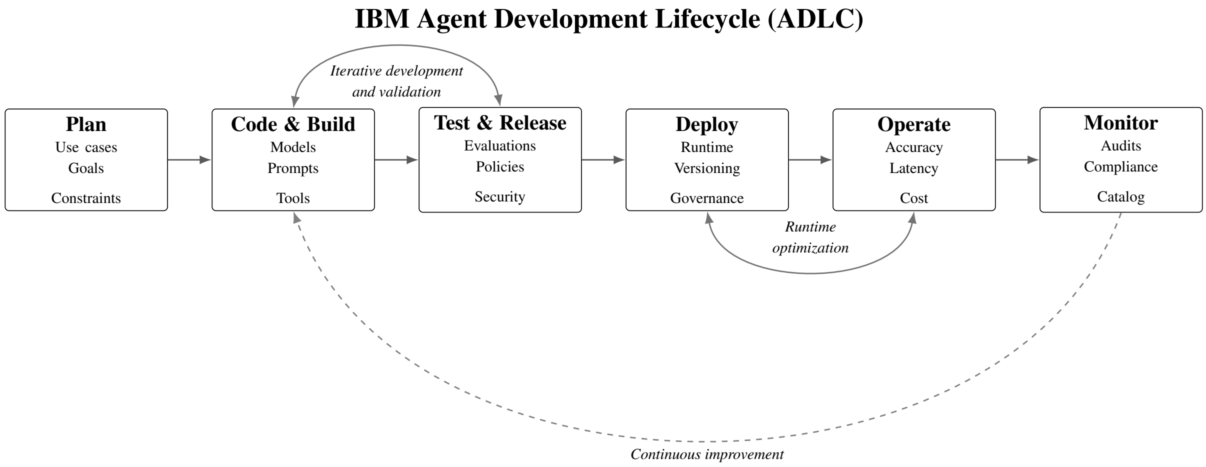

Figure 17: IBM Agent Development Lifecycle (ADLC)

In the VIP project, the Plan phase corresponds to defining agent use cases, expected behavior, schemas, and constraints. The Code & Build phase corresponds to configuring prompts, tools, models, and runtime artifacts. The Test & Release phase corresponds to validating agent outputs and schema compliance. The Deploy phase corresponds to packaging agents as Dockerized FastAPI services. The Operate and Monitor phases correspond to observing latency, token usage, cost, and execution traces.

IBM also describes agent building as the transformation of intent and design into concrete capabilities by configuring models, behavior, knowledge, and tools. This is aligned with the VIP approach, where agents are created from declarative configurations including prompts, schemas, tools, and model parameters.

1.7.5 Comparative Analysis of the Methodologies

The three methodologies were compared according to their relevance to the VIP project, especially with regard to flexibility, strategic vision, AI project support, agent lifecycle coverage, deployment, monitoring, and limitations.

Criterion	Scrum	IBM Data Science Methodology	IBM ADLC
Approach	Iterative and sprint-based	Structured and data/AI-oriented	Agent lifecycle-oriented
Flexibility	High	Medium to high	High
Strategic vision	Limited to product increments	Strong	Focused on agent operations
Fit for AI projects	Partial	Strong	Strong for agentic AI
Fit for agent lifecycle	Limited	Partial	Very strong
Deployment and monitoring support	Not detailed	Present through deployment and feedback	Central through deploy, operate, and monitor
Main limitation	Not enough governance for AI agents	Not specific enough for agent runtime behavior	Too agent-specific to structure the whole project alone

Table 1: Comparison of the methodological approaches considered for the VIP project

The comparison shows that none of the methodologies fully covers the needs of the VIP project when used alone. Scrum supports practical development organization, IBM Data Science Methodology provides the strategic project lifecycle, and ADLC addresses the agent-specific lifecycle required by configurable, deployable, and observable AI agents.

1.7.6 Adopted Hybrid Methodology for the VIP Project

Based on the comparison, the adopted methodology is a hybrid approach combining IBM Data Science Methodology and IBM ADLC, while keeping Scrum-inspired iterations for practical development organization.

IBM Data Science Methodology was selected as the main macro-framework because it structures the project from business understanding to deployment and feedback. It ensures that the project remains aligned with business objectives, technical constraints, and evaluation criteria.

ADLC was added because the VIP platform is centered on AI agents. The project required agent-specific phases such as planning agent behavior, configuring prompts and tools, validating outputs, deploying agents as services, operating them, and monitoring their execution.

Scrum was not selected as the main methodology because the project required stronger architectural governance and lifecycle control than a sprint-based framework alone could provide. However, Scrum-inspired practices were useful during implementation, especially for breaking the work into incremental tasks, validating features progressively, and adapting to feedback.

This hybrid methodology was therefore chosen because it provides a balance between structure, flexibility, and agent-specific lifecycle management. It supports the strategic organization of the project, the iterative improvement of AI components, and the operational requirements of an enterprise-grade agentic platform.

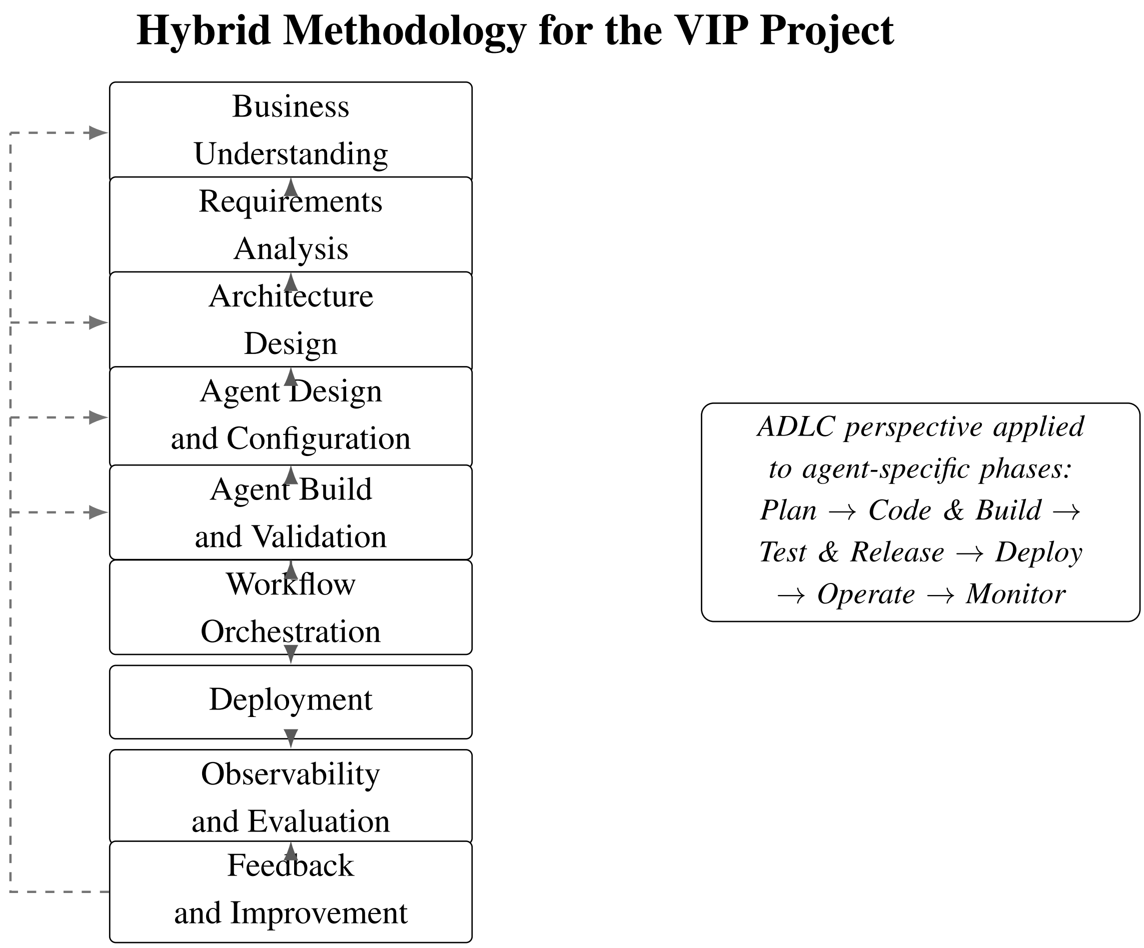

Figure 18: Hybrid methodology adopted for the VIP project

1.8 Project Timeline

The project officially started on December 1, 2025 and is scheduled to end on May 31, 2026, spanning a duration of six months. This timeframe was organized to support a structured and progressive approach, covering onboarding, research, design, development, integration, observability, and testing phases.

1.9 Summary

This chapter introduced the context of the internship by presenting the motivation behind enterprise AI agent orchestration, the host company Value Digital Services, and the objectives of the VIP project. It also clarified the main problem addressed by the project, namely the need for a modular, observable, cost-optimized, and provider-independent platform for creating and managing AI agents. The chapter then summarized the selected technologies, including FastAPI, Next.js, LangChain, LangGraph, Docker, MLflow, PostgreSQL, Laminar, and the LLM/SLM providers evaluated through SLM-evals, before presenting the methodological approach and the project timeline.

The next chapter focuses on the state of the art. It reviews existing agent frameworks, LLM providers, orchestration approaches, and evaluation methods in order to position the proposed solution with respect to current tools and practices.

Chapter 2

State of the Art

2.1 State of the Art of AI Agents and Orchestration

2.1.1 Defining AI Agents

An AI agent is a software entity capable of perceiving inputs, reasoning over context, invoking external tools, and producing structured outputs in order to achieve a defined objective. Unlike traditional prompt-response systems, agentic systems extend large language models (LLMs) with execution logic, memory, tool interfaces, and multi-step reasoning capabilities.

In enterprise environments, an agent is not merely a conversational component, but an operational unit that interacts with APIs, databases, and external systems. It may retrieve information, transform data, validate outputs, or trigger automated workflows. As a result, agents must be treated as managed execution components rather than isolated prompt wrappers.

This evolution from simple generation models to operational agents introduces new requirements: lifecycle management, configuration control, structured outputs, monitoring, and governance.

2.1.2 Single-Agent and Multi-Agent Systems

A single-agent system relies on one autonomous unit responsible for handling the entire task loop, including reasoning, tool invocation, and response generation. This approach is suitable for bounded or narrowly scoped tasks.

However, complex enterprise workflows often require decomposition into multiple responsibilities such as planning, retrieval, validation, transformation, and reporting. Multi-agent systems distribute these responsibilities across specialized agents, each with a defined role and structured input/output schema.

In such architectures, agents collaborate through intermediate outputs and shared state. While this improves modularity and reuse, it significantly increases coordination complexity. Consequently, multi-agent systems require orchestration mechanisms to manage sequencing, state transitions, schema compatibility, and error handling.

2.1.3 Why Orchestration Is Required

As agentic systems evolve from isolated assistants to collaborative execution networks, orchestration becomes a fundamental architectural requirement. Orchestration ensures that agents are executed in the correct order, that data flows between them in a structured manner, and that failures can be handled predictably.

Without orchestration, agent logic becomes embedded in ad hoc control code, making workflows difficult to reproduce, debug, or scale. Orchestration frameworks introduce structured patterns such as workflow graphs, state machines, retries, and task delegation.

In enterprise contexts, orchestration is not only about execution sequencing. It also ensures traceability, auditability, and governance of agent interactions.

Figure to be added later.

Figure 19: Evolution from single-agent execution to orchestrated multi-agent workflows.

2.1.4 Provider Landscape: Frontier LLMs and Small Language Models

The provider layer is a strategic dimension of agentic systems. Frontier LLMs such as GPT-series or other commercial APIs offer strong reasoning performance and ease of integration. However, they introduce dependency on external infrastructure, token-based operational costs, and potential vendor lock-in.

Small Language Models (SLMs) and open-weight models offer an alternative path. They may be self-hosted or deployed closer to the application environment, improving latency control and governance. However, their suitability depends on rigorous evaluation against task-specific requirements.

For enterprise platforms, provider flexibility becomes essential. A robust architecture should abstract the provider layer, enabling model replacement based on cost, latency, compliance, or performance constraints.

2.2 Preliminary Study and Project Specifications of the VIP Project

2.2.1 Problem Statement

Despite the rapid evolution of agentic AI frameworks and language models, enterprise deployment remains fragmented. Existing solutions often provide partial capabilities, focusing either on workflow automation, agent collaboration, or managed cloud execution.

However, organizations require platforms that combine:

- Configuration-driven agent generation
- Provider flexibility

- Workflow orchestration
- Deployment automation
- Observability and governance

The central challenge addressed by VIP is therefore:

> How can we design a modular, observable, provider-neutral AI agent orchestration platform capable of generating and managing agents dynamically while maintaining cost control, scalability, and governance?

2.2.2 Business Objectives

The business objectives of VIP are:

- Reduce vendor lock-in by abstracting model providers
- Enable rapid agent creation without modifying source code
- Improve governance through versioning and traceability
- Reduce operational cost through modular deployment
- Provide reusable workflows for domain-specific AI applications

2.2.3 Technical Objectives

From a technical perspective, the VIP project aims to provide a modular platform for creating, deploying, orchestrating, and supervising AI agents. The platform must support configurable agent behavior, reusable workflow composition, provider abstraction, and structured communication between agents.

These objectives are not limited to model performance. They also concern the way agents are generated, connected, monitored, and maintained throughout their lifecycle. Therefore, the main technical objectives of VIP are to enable configuration-driven agent creation, standardize agent communication through schemas, support multiple LLM and SLM providers, and ensure traceability during execution.

The model evaluation aspect is supported by the SLM-evals second project, which is presented separately in Section 2.6.

The main technical objectives of VIP are:

- enable dynamic agent creation from declarative configuration files;
- support reusable multi-agent workflows;
- provide a unified provider abstraction layer;
- ensure structured input and output validation through schemas;
- support prompt and schema versioning;
- provide observability over agent execution, token usage, latency, and errors;
- allow future integration of model evaluation results into provider selection.

2.2.4 Functional and Non-Functional Requirements

2.2.4.1 Functional Requirements

2.2.4.2 Non-Functional Requirements

2.2.5 Success Criteria

2.3 Comparison of Existing Platforms with Respect to VIP

2.3.1 Evaluation Criteria Derived from VIP Requirements

2.3.2 Comparative Study of n8n, CrewAI, and Jarvio

2.3.3 Discussion of Platform-Level Gaps

2.4 Evaluation and Operational Foundations

2.4.1 Evaluation Frameworks for Agentic Systems

2.4.2 Observability and Governance Requirements

2.4.3 Why Observability Is a First-Class Need for VIP

2.5 Synthesis and Gap Analysis for the VIP Project

2.5.1 Second Project Preliminary Study (SLM-Evals)

2.6 Chapter Summary